

TrustLens Viva Prep

Tech Stack Basics — Q&A;

What we built it with, and why each choice — separate from the Day 1/2/3 files

How this file is organised

Every question has a small coloured tag. **Green "BASIC"** is the kind of question asked to check you understand your own project at all — don't skip these, a panel asks them first and marks confidence as much as correctness. **Red "TOUGH"** goes deeper: trade-offs, alternatives, and "why not X instead". Read every BASIC one at least once out loud before the panel — they feel too easy to prepare for, which is exactly why people fumble them.

One honesty rule for this file

Everything below names a real library actually installed in this project (checked directly in the backend's virtual environment before writing this), not a guess. If a panel member asks about a technology NOT in this file, the safe honest answer is "we didn't use that in this project" — never invent familiarity with a tool you didn't touch.

1. The Whole Stack, in One Table

"Stack" just means: which pieces of software does the project run on, stacked in layers from what the user sees down to where the data lives. Say this table out loud once, in order, before anything else.

Layer	What we used	One-line reason
User's browser	React 19 + Vite	Builds the interface as reusable components; Vite is the tool that runs and bundles it fast.
Talking to the server	fetch() (built into the browser)	No extra library needed for HTTP requests — the browser already has one.
Page navigation	React Router 7	Lets the app feel like multiple pages (Login, Dashboard, Results) without a full page reload each time.
Server / API	FastAPI (Python)	Turns Python functions into web endpoints (like /analyze-live) automatically, with built-in validation.
Server process	Uvicorn	The actual program that keeps FastAPI running and listening for requests.
Database	SQLite, via SQLAlchemy	One file on disk, no separate database server to install; SQLAlchemy is the Python layer that talks to it.
Login security	bcrypt + PyJWT	bcrypt turns a password into an unreadable hash; JWT is the signed "ticket" a logged-in browser holds.
Fake-account AI	scikit-learn (Random Forest)	A well-understood, explainable model — good fit for structured numeric profile data.
Misinformation AI	PyTorch + Hugging Face Transformers (XLM-RoBERTa)	A pretrained language model fine-tuned on our own labelled text data.
Getting Instagram data	RapidAPI (a third-party service)	Instagram's own official API needs a lengthy business approval process we didn't have time for.

The one metaphor that explains "frontend vs backend"

A restaurant: the **frontend** is the dining room and the menu — what the customer sees and touches. The **backend** is the kitchen — where the real work happens, out of sight. The **database** is the pantry/fridge — where ingredients (data) are stored between uses. The customer never walks into the kitchen directly; they always go through a waiter. In our app, that "waiter" is the API call the frontend makes to the backend.

2. Frontend Basics

BASIC Q: What is "frontend"?

Everything that runs inside the user's own browser: what they see, click, and type into. It has no direct access to the database — it can only ask the backend for things.

BASIC Q: What is React?

A JavaScript library for building user interfaces out of small, reusable pieces called "components" (e.g. the score ring, the navigation bar). Instead of writing one giant page, you write small pieces and combine them.

BASIC Q: What is a "component"?

A self-contained, reusable chunk of interface with its own logic — like `ScoreRing.jsx`, which draws the Trust Score circle and can be reused anywhere a score needs to be shown.

BASIC Q: What is Vite, and how is it different from React?

React is the library that describes WHAT the interface looks like. Vite is the tool that takes all our source code files and turns them into the actual files a browser can run, and serves them during development with instant reload.

BASIC Q: What is JSX?

A way of writing HTML-like markup directly inside JavaScript code, e.g. `<h1>{score}</h1>`. It looks like HTML but is actually converted into JavaScript function calls before running.

BASIC Q: What is "state" in React?

Data that a component remembers between renders and that can change over time — like the username someone has typed into the scan box before pressing submit.

TOUGH Q: Why React and not just plain HTML/CSS/JavaScript?

Our app has real interactive, data-driven pages (a live scan that updates itself, a dashboard that filters in real time, a score that animates in). Doing that by hand in plain JS means manually finding and updating page elements everywhere data changes, which gets messy fast. React handles that update logic for us: when data changes, React figures out what changed on screen.

TOUGH Q: Why React and not Vue or Angular?

All three would work. React has the largest community and the most learning material, which mattered for a first-time frontend developer building this project mostly solo. It's also the most common one asked about in industry interviews, so what was learned here transfers.

TOUGH Q: What is a Single Page Application (SPA), and is that what this is?

Yes. A SPA loads one HTML page once, and JavaScript (React Router) swaps out what's shown as the user navigates, instead of asking the server for a whole new page each time. It feels instant, at the cost of a slightly heavier initial load and worse out-of-the-box search-engine visibility (not a concern for a login-gated tool like this).

TOUGH Q: What is React Router doing, concretely?

It watches the browser's address bar (e.g. `/dashboard`, `/results?id=4`) and decides which component to show, without asking the server for a new page. Clicking a link just swaps components in place.

TOUGH Q: What does "the frontend calls an API" actually mean, mechanically?

The browser's built-in `fetch()` function sends an HTTP request (e.g. POST to `http://127.0.0.1:8000/analyze-live`) with the username as JSON data, waits for a response, and the code then reads the JSON back and updates the page with it.

3. Backend Basics

BASIC Q: What is "backend"?

The part of the app that runs on a server (not in the user's browser). It holds the real logic and secrets (like how the Trust Score is calculated, and the database password), and the frontend is never allowed to do that work itself.

BASIC Q: What is Python, and why is it used here instead of JavaScript on the backend too?

Python is the dominant language for machine learning — the libraries our AI models need (PyTorch, scikit-learn, Transformers) are Python-first. Using Python for the backend means the same language runs the web server AND the AI models, with no need to bridge two different languages.

BASIC Q: What is an API?

A defined set of "doors" a program exposes so other programs can ask it to do things — for us, e.g. POST /analyze-live is a door that takes a username and returns a Trust Score.

BASIC Q: What does "endpoint" mean?

One single API "door" — one URL path plus one HTTP method, like GET /scans or POST /login. Our backend has around 20 of these.

BASIC Q: What is JSON?

A simple text format for structuring data with keys and values, e.g. {"username": "cristiano", "trust_score": 77.4}. It's how the frontend and backend send data to each other — both JavaScript and Python can read and write it easily.

BASIC Q: What is FastAPI?

A Python framework for building web APIs. You write a normal Python function, put a decorator like @app.post("/login") above it, and FastAPI turns it into a real web endpoint — it also automatically checks that incoming data has the right shape.

TOUGH Q: Why FastAPI and not Flask or Django?

FastAPI is built for APIs specifically (Flask/Django were originally built for full websites with server-rendered pages). It has automatic request validation, is fast to write, and it auto-generates interactive API documentation at /docs, which we used constantly while testing the backend by hand.

TOUGH Q: What is Uvicorn, and why do we need it if we already have FastAPI?

FastAPI describes WHAT each endpoint does. It doesn't, by itself, listen on a network port and accept real HTTP connections — that's Uvicorn's job. FastAPI is the recipe; Uvicorn is the kitchen that actually cooks it and serves it to browsers.

TOUGH Q: What does "async" mean, and does this project use it?

Async lets a program start one slow task (like calling an external API) and go do other work while waiting, instead of freezing until it finishes. Our /analyze-live endpoint uses a ThreadPoolExecutor to fetch the profile and the engagement data from RapidAPI at the same time rather than one after another, which is the main reason a scan got noticeably faster.

TOUGH Q: What is CORS, and why did we have to configure it?

Browsers block a webpage from one address (e.g. localhost:5173, the frontend) from calling an API on a different address (localhost:8000, the backend) unless the API explicitly allows it. CORS ("Cross-Origin Resource Sharing") is that permission system. main.py has a CORSMiddleware block that allows it, otherwise every fetch() call from the frontend would be silently blocked by the browser.

TOUGH Q: What is a "dependency" in FastAPI (e.g. `Depends(get_current_user)`)?

A reusable piece of setup code that FastAPI runs automatically before your endpoint's own code, and hands you the result. `get_current_user` reads the Authorization header, checks the login token, and gives your function the logged-in user object — so that check doesn't need to be copy-pasted into every single endpoint.

4. Database Basics

BASIC Q: What is a database?

A structured place to permanently store data, so it's still there after the program restarts. Without one, every scan and every user account would vanish the moment the server stopped.

BASIC Q: What is SQLite?

A database that lives in a single file on disk (backend/trustlens.db) rather than needing a separate program running in the background. It's the simplest real database to set up, which mattered a lot for a beginner's first project.

BASIC Q: What is SQL?

The standard language for asking a database questions or telling it to change something, e.g. `SELECT * FROM users WHERE email = ...`. SQLite understands SQL.

BASIC Q: What is a "table", a "row", and a "column"?

A table is like one spreadsheet (e.g. the `scan_history` table). A row is one entry in it (one saved scan). A column is one field every row has (username, trust_score, verdict, and so on).

BASIC Q: What is an ORM, and what is SQLAlchemy?

ORM = Object-Relational Mapper. Instead of writing raw SQL text by hand everywhere, you write normal Python code (e.g. `db.query(User).filter(...)`) and the ORM translates it into SQL for you. SQLAlchemy is the specific ORM library this project uses.

TOUGH Q: Why SQLite and not PostgreSQL or MySQL?

Scope and pragmatism: SQLite needs zero installation or server management, which is ideal for a student project on one machine. The code is written so this isn't a dead end — `database.py` reads a `DATABASE_URL` environment variable, and switching to PostgreSQL later would need changing that one setting, not rewriting the queries, since SQLAlchemy abstracts over both.

TOUGH Q: What happens if two people scan at the exact same time — does the database break?

SQLite handles one write at a time internally (it locks briefly during a write), so it won't corrupt data, but it isn't built for many simultaneous heavy writers the way PostgreSQL is. For this project's real usage pattern (a handful of people testing at once), that's not a practical problem.

TOUGH Q: How does the database schema get created or updated?

`database.py`'s `init_db()` runs `Base.metadata.create_all()` on server start, which creates any missing tables. A separate helper, `_add_missing_columns()`, runs `ALTER TABLE ADD COLUMN` for any new columns on tables that already exist, so an old copy of the database file doesn't crash when the code adds a new field — it was actually needed and tested when the admin console's new columns (role, `is_active`) were added.

TOUGH Q: What is a foreign key, and where do we use one?

A column in one table that points to a row in another table, enforcing that the link is valid. `ScanHistory.user_id` points to `Users.id` — every saved scan is linked to the account that ran it (or null, if run while logged out, before this session's login-required change).

TOUGH Q: What's the difference between the ORM model (e.g. class `User`) and the raw table?

The class (in `database.py`) is the Python-side description of the table's shape — it's what the code actually reads and writes. SQLAlchemy translates operations on that class into the real SQL statements that hit the actual table in the `.db` file.

5. Login & Security Basics

BASIC Q: Why can't the password just be stored as plain text in the database?

If the database was ever leaked or accessed by an attacker, every user's real password would be exposed immediately — and since many people reuse passwords, that risk extends to their other accounts too.

BASIC Q: What is "hashing"?

A one-way transformation: you can turn a password into a hash easily, but you (practically) cannot turn the hash back into the original password. Logging in checks "does hashing this typed password produce the same hash we stored?" rather than ever comparing raw passwords.

BASIC Q: What is bcrypt?

The specific hashing algorithm this project uses for passwords. It's deliberately slow (by design), which makes it expensive for an attacker to try guessing millions of passwords against stolen hashes.

BASIC Q: What is a "token", and what is JWT?

After logging in successfully, the server gives the browser a token — a signed piece of text proving "this person really did log in". JWT (JSON Web Token) is the specific, standard format that token is written in. The browser sends it back on every later request instead of the password.

TOUGH Q: How does the server know a JWT token wasn't faked or edited by the user?

The token is cryptographically signed with a secret key (JWT_SECRET) that only the server knows. Any change to the token's contents (e.g. editing the user ID inside it) breaks the signature, and the server's check (`jwt.decode(...)`) rejects it immediately.

TOUGH Q: Why JWT instead of traditional server-side sessions with cookies?

A JWT is self-contained — the server can verify it just by checking the signature, without looking anything up in the database on every single request. A traditional session needs a server-side session store to check against. For a small project this is a simpler mental model and needs no extra infrastructure.

TOUGH Q: What stops an expired or stolen token from being used forever?

Every token has a 7-day expiry baked into it (TOKEN_EXPIRES_HOURS). After that, `jwt.decode()` raises an expired-signature error and the request is rejected, so the user has to log in again.

TOUGH Q: What is a "salt", and does bcrypt use one?

A salt is random data mixed into a password before hashing, so the exact same password produces a different hash for every user. This stops an attacker from pre-computing hashes for common passwords once and reusing that list against every account (a "rainbow table" attack). `bcrypt.gensalt()` generates a fresh random salt automatically for every password hashed.

TOUGH Q: Is this project's login 2FA (two-factor authentication)?

No, and it's important to say that precisely if asked. It's one-time email verification at signup — the email is proven real once, then login is password-only forever after. A genuine 2FA-at-login design was actually built first and deliberately replaced, because it made no sense for this kind of app to demand a fresh code on every single login.

6. AI / Machine Learning Basics

BASIC Q: What is "machine learning", in one sentence?

Instead of a human writing exact rules for every situation, you show the computer many labelled examples (this account is fake, this one is real) and it works out its own rules from patterns in the data.

BASIC Q: What is a "trained model"?

The end result of that learning process — a file that has learned patterns from the training data and can now make predictions on new, unseen data. `fake_follower_model.pkl` is our trained model file.

BASIC Q: What is scikit-learn?

A Python library with ready-made implementations of classic machine learning algorithms (including Random Forest, which this project uses for fake-account detection). You don't have to write the algorithm's maths yourself.

BASIC Q: What is a Random Forest, in plain words?

Many simple decision trees (each one asking a series of yes/no questions like "does it have a profile picture?") are trained slightly differently from each other, and their individual votes are combined into one final answer. Combining many imperfect "opinions" tends to be more accurate than trusting just one.

BASIC Q: What is a "held-out test set", and why does it matter?

A portion of labelled data the model is never shown during training, kept aside purely to check accuracy honestly afterwards. A number measured this way is trustworthy; a number measured on the training data itself is not, because the model may have simply memorised those exact examples.

TOUGH Q: Why Random Forest for fake-account detection, and not a neural network?

The input data here is a small number of clean, structured numbers (follower count, post count, bio length, etc.) — exactly the kind of data classic models like Random Forest already handle very well. A neural network typically needs far more data to outperform simpler models on this kind of small structured/tabular dataset, and would be much harder to explain to a panel ("the model made 200 trees vote" is a far more defensible answer than "the neural network's hidden layers did something").

TOUGH Q: What is PyTorch, and why is it needed alongside scikit-learn?

PyTorch is a deep learning framework, used for building and running neural networks — needed for the misinformation classifier (XLM-RoBERTa), which is a large, pretrained transformer language model. scikit-learn's simpler algorithms (like Random Forest) don't need a deep learning framework at all, so both libraries are used, each for the job it fits.

TOUGH Q: What is Hugging Face Transformers?

A library that gives easy access to thousands of pretrained language models (including XLM-RoBERTa) and handles loading them, tokenising text into a model-readable format, and running predictions — without having to build a transformer model completely from scratch.

TOUGH Q: What does "fine-tuning" a model mean, and did we do it?

Starting from a model that's already been pretrained on huge amounts of general text (so it already understands language broadly), then continuing to train it further on our own specific, smaller, labelled dataset (misinformation examples) so it specialises for our exact task. Yes — the misinformation classifier is XLM-RoBERTa, fine-tuned on our own 46,392-row labelled dataset, not trained completely from zero.

TOUGH Q: What is "overfitting", and how do we know our models aren't doing it?

When a model performs well on the data it was trained on but poorly on new data, because it memorised specifics instead of learning general patterns. We guard against reporting an overfit number by only quoting accuracy measured on a genuine held-out test set the model never saw during training (95.8% and 77% for our two real trained models).

TOUGH Q: Are the Engagement Analyzer and Credential Extractor "AI"?

No, and this must never be blurred. Both are rule-based / pattern-matching logic (if-statements and regular expressions), not trained models. Only the fake-account detector (Random Forest) and the misinformation classifier (XLM-RoBERTa) are genuine trained AI in this project.

7. Getting Real Data In: RapidAPI & Email

BASIC Q: How does the app get real Instagram data?

It calls a third-party service called RapidAPI, which itself calls Instagram and hands back structured data (follower count, posts, bio, etc.) as JSON. Our backend never talks to Instagram directly.

TOUGH Q: Why RapidAPI and not Instagram's own official Graph API?

Meta's official Graph API requires a formal business verification/review process that takes weeks and needs a registered business behind it — not realistic on a student project's timeline. RapidAPI gives access to the same kind of public data, faster to integrate, as a documented pragmatic trade-off.

TOUGH Q: What is a "rate limit", and did we hit one?

A cap a service places on how many requests you're allowed to make in a period of time. Yes — the free RapidAPI tier is 20 requests/month (not per day, a real early misunderstanding that was corrected after checking the dashboard directly), which made every real test scan during development something to ration carefully.

BASIC Q: How does the app send the sign-up verification code by email?

Through Gmail's own SMTP (email-sending) server, using a Gmail account's address and an app-specific password, from backend/email_service.py.

TOUGH Q: What is SMTP?

The standard protocol email programs use to actually send an email across the internet. Our backend acts as a simple email client, logging into Gmail's SMTP server and asking it to deliver the verification code.

8. If the Panel Asks Something Not on This List

The one safe answer

"I haven't used that specific tool in this project, so I'd rather not guess — but here's the closest thing we did use and why." Then bridge to whatever real choice you made instead. This reads as honest and in control, not as a gap. Never claim to have used something you didn't; it's the fastest way to lose credibility if a follow-up question exposes it.

This file is separate from Day 1/2/3 — go back to those for module-specific and code-walkthrough prep.